SMART ANALYSIS REPORTER

Technical & System Report

System Documentation

Student Performance Prediction

Prepared by Kavin Ganapathy
Student ID: 100008820
Date: 2026-06-07

Table of Contents

1. Introduction & Purpose

1.1 About this document

This document describes the design and implementation of **Smart Analysis Reporter**, a web-based statistical analysis and reporting toolkit. It is intended for evaluators and future maintainers, and explains what the system does, how it is built, the technologies it uses, and the results it produces on its demonstration dataset.

1.2 Problem the system solves

PROBLEM STATEMENT

Producing a statistical report normally means repeating the same manual steps — loading data, running tests, making charts, writing up findings — for every dataset. Smart Analysis Reporter automates that pipeline: it analyses a dataset and assembles an editable, exportable analytics report with minimal effort.

1.3 Scope

- Interactive exploration: profiling, descriptive statistics, frequency tables, Q-Q plots, correlation, hypothesis testing and regression.
- Automated reporting: one-click report generation with editable narrative.
- Export: PDF, Word and HTML deliverables.

2. Abstract

Smart Analysis Reporter loads a dataset, runs a battery of descriptive and inferential analyses, and assembles an editable analytics report that exports to PDF, Word and HTML. This document demonstrates the system on the **Student Performance Prediction** dataset (10,000 rows × 8 columns), whose objective is to understand and predict student exam performance. The toolkit is organised as a layered, modular Python application with a Streamlit front end and a dedicated report engine.

3. System Design

3.1 Architecture

The application follows a layered, modular design with a clear separation of concerns. A thin data layer caches the dataset; pure-Python analysis modules implement the statistics independently of the UI; a Streamlit multipage front end exposes them interactively; and a report engine converts any analysis into embeddable charts, tables and narrative.

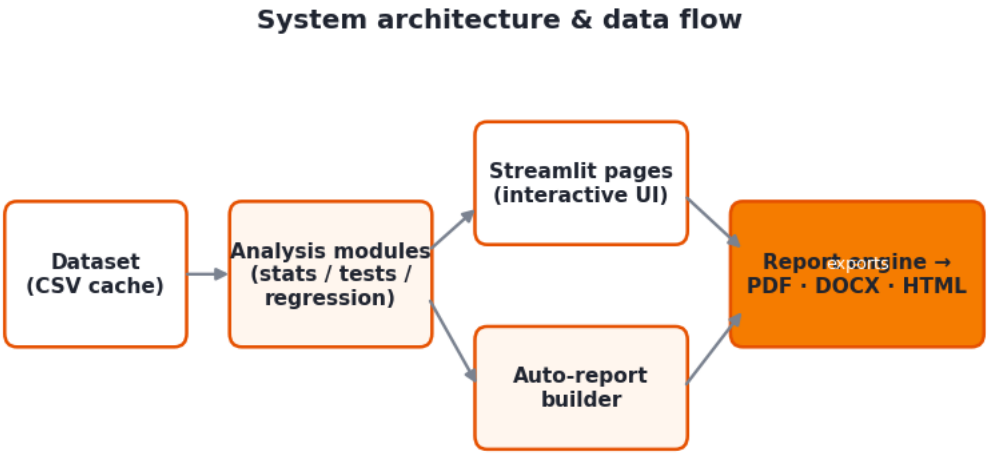

Figure 3.1 — System architecture and data flow.

3.2 Data flow

- The dataset is loaded once into session state and shared across all pages.
- Each analysis page calls the relevant module and renders interactive Plotly charts.
- An “Add to report” action captures any chart/table plus an auto-written inference.
- The auto-report builder chains the modules to assemble a full report in one click.
- The report engine renders themed Matplotlib images and exports PDF / DOCX / HTML.

3.3 Design principles

- Modularity — analysis logic lives in importable modules, not in the UI.
- Reusability — the same chart and inference helpers power the app and these documents.
- Portability — the dataset is cached in-repo so the deployed app needs no credentials.

4. Technology Stack

The toolkit is built entirely in Python with the following components:

Layer	Technology	Purpose
UI	Streamlit	Multipage interactive web app & widgets
Computation	pandas, NumPy	Data handling and vectorised computation
Statistics	SciPy, statsmodels	Statistical tests, distributions, OLS regression
Interactive charts	Plotly	Interactive on-screen charts
Report charts	Matplotlib	Themed images embedded in exported documents
Export	ReportLab, python-docx	PDF and Word document generation
Data source	kagglehub	Fetching the demonstration dataset

5. Dataset Introduction

The **Student Performance Prediction** dataset contains 10,000 student records with 8 variables (7 metric, 1 categorical) and 0 missing values. The target variable is **exam_score**. The full schema is:

Column	Type	Example	Unique
study_hours	Metric	7	11.000
attendance	Metric	56	61.000
sleep_hours	Metric	8	6.000
internet_usage	Metric	7	11.000
assignments_completed	Metric	10	21.000
previous_score	Metric	62	61.000
exam_score	Metric	100.0	3563.000
placement_status	Categorical	Placed	2.000

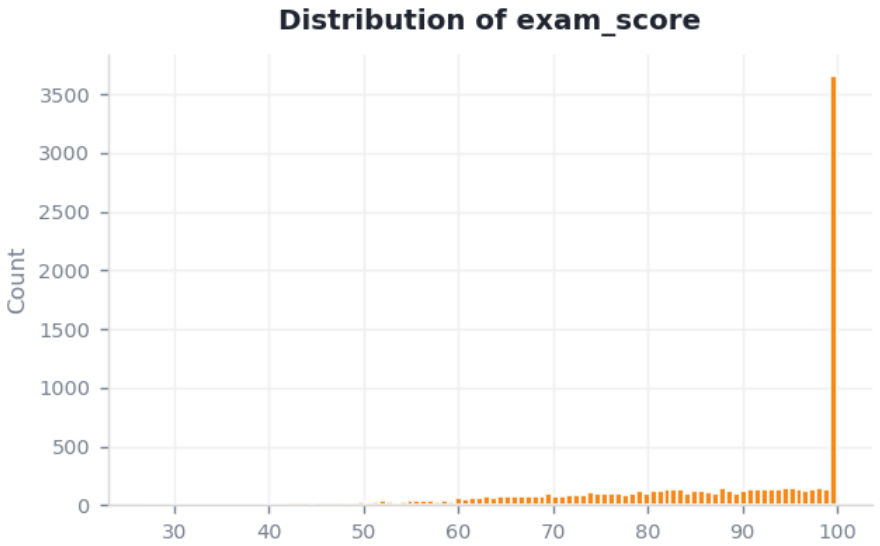

Figure 5.1 — Distribution of the target variable, exam_score.

6. Analysis

The toolkit applies descriptive statistics, frequency analysis, normality checks (Q-Q plot + Shapiro-Wilk), correlation, hypothesis tests (t-tests, ANOVA, chi-square, Z-tests) with live assumption checks, and linear regression. Two representative outputs are shown below.

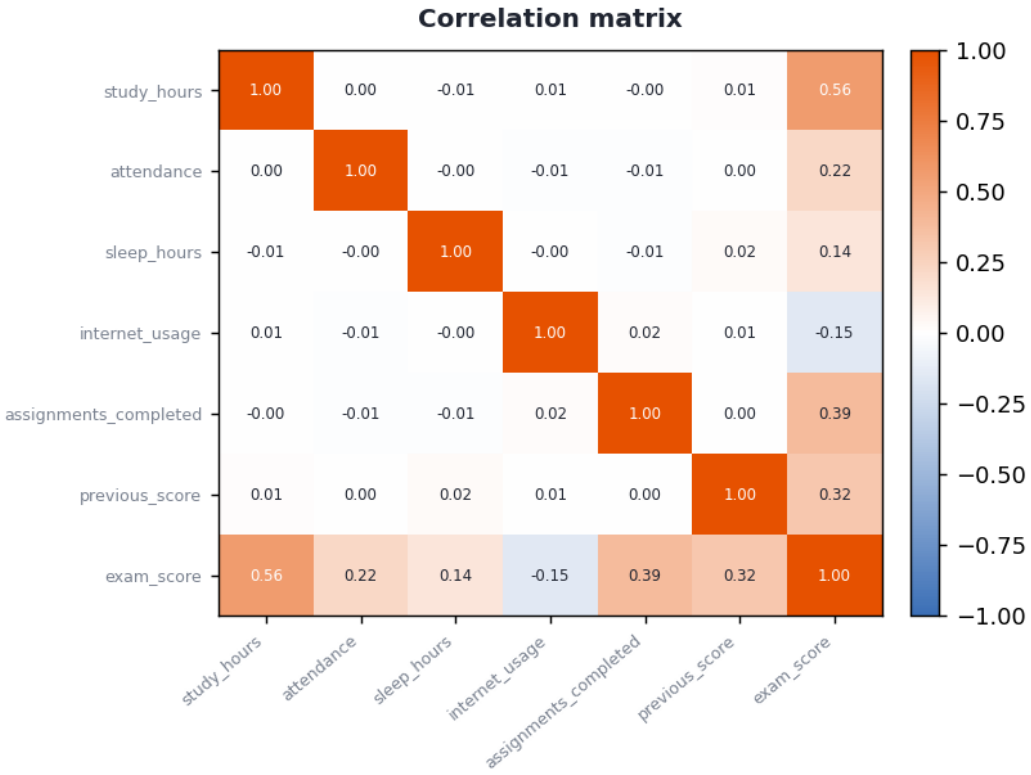

Figure 6.1 — Correlation matrix of the metric variables.

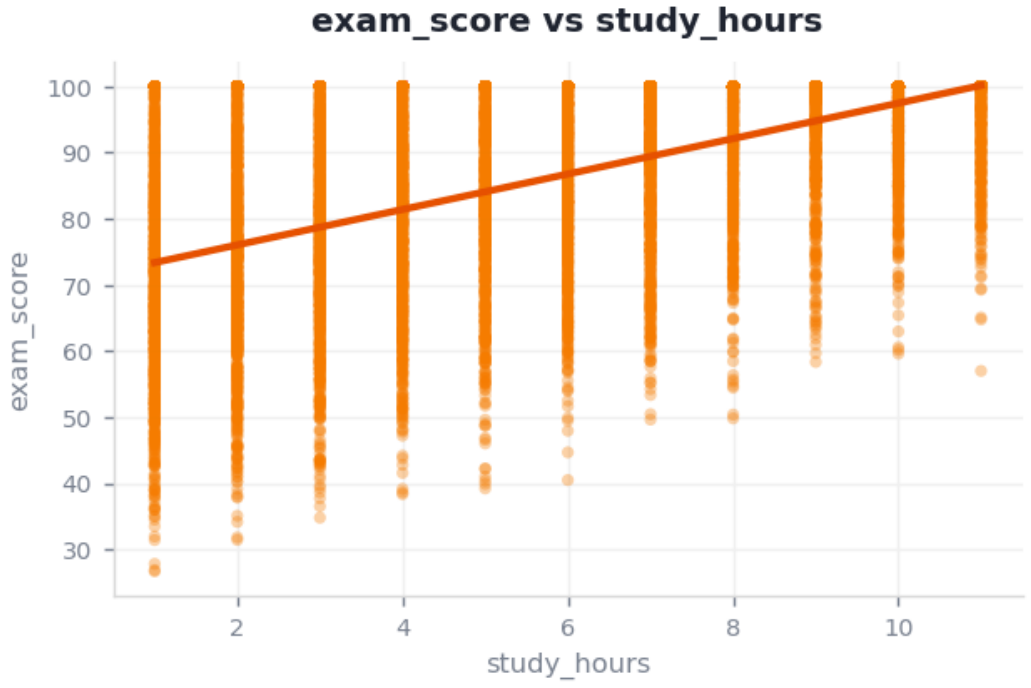

Figure 6.2 — Linear regression of exam_score on study_hours.

7. Conclusion of Analysis

Applied to the demonstration dataset, the toolkit produced the following evidence-based findings:

- The strongest linear relationship is between **study_hours** and **exam_score** (r = 0.56).
- A simple model of **exam_score** on **study_hours** explains 31.6% of its variance ($R^2 = 0.32$).
- There is a statistically significant difference in **exam_score** between **placement_status** groups ($p < 0.001$).
- **exam_score** is not normally distributed (Shapiro-Wilk W = 0.8376).

8. Project Conclusion

Smart Analysis Reporter demonstrates an end-to-end analytics workflow — from raw data, through an interactive toolkit, to a polished and editable report — without hand-coding each study. Its modular structure means new analyses or datasets can be added with minimal change, and its one-click reporting makes findings immediately shareable. The Student Performance Prediction case study shows the toolkit producing a complete, defensible statistical narrative automatically.

8.1 Future work

- Multiple-regression and classification models for richer prediction.
- Automated outlier and data-quality reporting.
- User accounts and saved report templates.